Cheatsheet

Calling convention and Registers

Calling Convention 包括：参数、返回值/地址、传参、出入栈、fp 的建立和还原、调用方/被调用方保存的寄存器。

调用的约定

在机器的宏观视角下，函数/过程调用的流水线：

从调用方 (Caller) 开始…

1. **准备参数**（寄存器不够/需要按约定放栈时就放栈上）
2. **保存返回地址**（ret）
3. **跳转到被调函数**

工作移交到被调用方 (Callee)…

4. **建立栈帧 (Stack frame)**：栈指针下移，给调用方的栈帧/局部变量/保存寄存器留空间（sp -= stackFrame）

5. **保存需要保存的寄存器**

6. **执行函数主体**

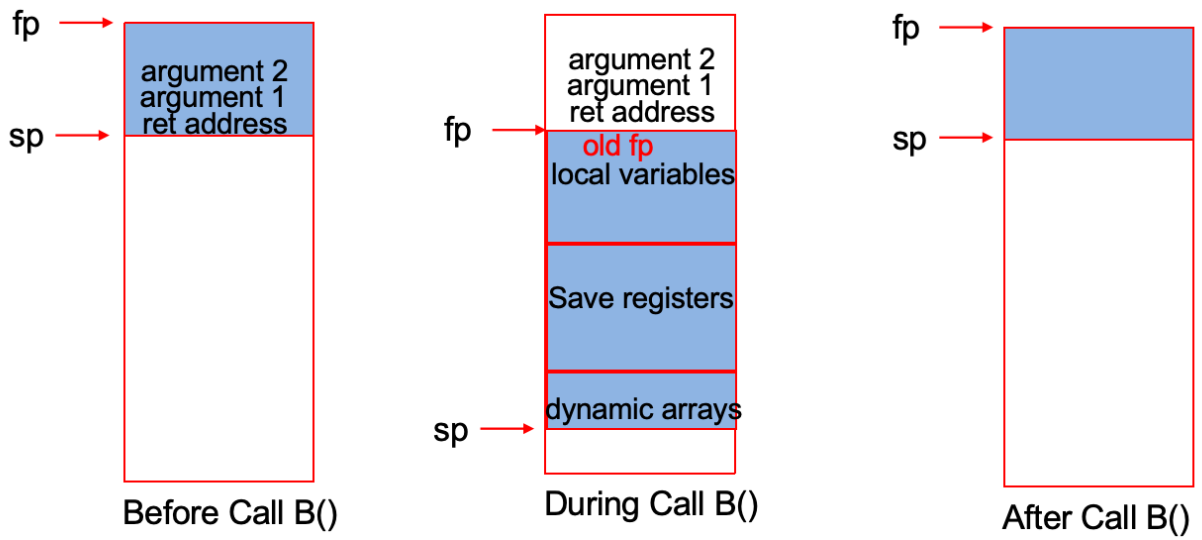
7. **准备返回值**（可能放寄存器 a0/a1，也可能放栈）

8. **恢复保存的寄存器 **并 **返回调用方位置**

- ⚠ **“栈帧”** 是栈上分配给函数的私有区域，每个函数有自己独立的一段帧 (向下分配)；**"帧指针 fp"** 是函数栈帧的寻址基点，这里用于返回后将栈变回调用前的原样。

Meanwhile，栈身上发生了什么？

1. 第 1～3 步 (处于调用方) 时，调用者**建立自己的栈帧**将参数 (如果 a0~a7 放满了) 和返回地址入栈；
2. 第 4～7 步 (处于被调用方) 时，被调用者**建立新栈帧**，将局部变量、要保存的寄存器 (saved regs)、动态对象入栈到这个帧中；
3. 第 8 步 (回到调用方) 后，**被调用者的栈帧被出栈**，栈回到第 1 步之前的样子，fp/sp 指向原先位置。



"Saved Regs"

ABI 规定了一些需要分工在栈中保存下来的寄存器。防止连环调用时关系混乱导致变量覆盖或生命周期错误。逻辑上类似于——

“单位的电脑属于信息部 (Caller)，但所有电脑都有信息/公关两部门的账号。公关部 (Callee) 帮信息部的忙，借用电脑后要负责把账号切回信息部 (Callee-saved)；而信息部负责留档自己电脑上的重要文件 (Caller-saved)。”

A) Callee-saved (被调用者必须保存)

- **s0–s11** (saved regs)、 (old) **fp —— 常用 s0 (即 x8) 充当 fp**、**CSEs**。
- 比如：**跨多个 function call 仍保留的变量**、**长寿命 local variables**、循环里长期使用且中间会 call 别的函数的值、**函数状态 / 累积值 / base pointer / 长期活跃指针**、**Leaf Routines (叶子函数)**

规则：长期/跨调用——**B 如果要用/改这些寄存器，必须先保存旧值，返回前恢复**。

→ 这些被调用者存到栈里的备份，就属于 **saved regs**，一般生命周期较长。

B) Caller-saved (调用者必须保存)

- **t0–t6** (临时变量)、**a0–a7** (参数)、**ra** (返回地址)。
- 比如：**expression evaluation temporaries**、短期中间结果、**一次计算链上的临时值**、**只在 call 之前/之后短时间用到的值**、**不需跨函数调用保存的临时变量**、参数传递用值、Non-leaf Routines (非叶函数)

规则：临时/不跨调用——**B 可以随便改**。A 如果在 call 之后还要用，就必须自己先存起来。

→ 一般生命周期较短。

RV Registers	ABI registers	Purpose
X1	ra	Return address
X2 (sp)	sp	Stack top
X3	gp	Global pointer
X8 (fp)	fp	Frame pointer
X16-X18		Intra-procedure call scratch reg
X10-X17	a0-a7	Arguments
X5-X7, X28-X31	t0-t6	Caller Save
X8-X9, X18-X27	s0-s11	Callee Save

补充：x0 永远是 0，所以永远不必保存

Processor Architecture and Logic Design

> 指令到底是怎么在处理器里跑起来的？

Week 5 计算机架构：ISA (x86, ARM, RISCV…)；Week 6 处理器微架构：同一 ISA 下的不同实现 (Single-cycle、Multi-cycle、Pipelined、Out-of-Order…)

处理器微构架

- Data Path：运算、处理或搬运数据，负责数据流
- Control：指令的更新、抓取，负责指令流

Cycle

- CPU 并非连续运作，而是按照 “fetch, decode, execute” (从内存/缓存取指、用 ISA 解译命令、运算或访问)的循环周期性工作。最小时间单位为 cycle (不同实现时长不同)，以 GHz (频率)计算，比如 3GHz 的 CPU每秒处理 $3x10^9$ 个 cycle。
- 取指令 、拆开指令字段并根据 ISA 解释、根据不同指令类型执行不同运算 (然后可能需要访存、写回寄存器、最后**PC += 4**)。(⚠ RV32I 的 **C-extension** 情况下命令宽为“可变长” **16b/32b**，可能需要 PC += 2)

单周期：**每条指令都必须在一个周期内完成。**导致周期**向上对齐**——所有指令和最慢的 (即 `lw`) 一样耗时。(最快的命令是 Jump，比如 `jal`)

多周期：**把一条指令拆成多个阶段，每个阶段一个周期。**RISC 推动了周期缩短的革命，多周期执行明显更高效。

CPU 性能公式

⚠ *CPU Time = Instruction-Count x CPI x Clock-Cycle-Time*

即：命令的执行用时 = 需要执行多少指令 x 每条指令平均用多少周期 x 周期时长

- Instruction count 受 ISA、compiler、具体算法的影响；
- CPI 受 ISA、处理器微架构的影响；
- Clocktime 受处理器微架构、硬件技术的影响。

⚠ 判断性能需要同时考虑三者而不是只看执行多少指令！比如使用包含 FMA 的 ISA 导致 Instruction count 减小，若是 Single-cycle，CPI 仍为 1 但 Clocktime 变大；若是 Multi-cycle 则 CPI 变大但 Clocktime 不变。

总体归结为：**CISC (短、长、长) vs. RISC (长、短、短)** —— 故以 x86 为例的现代高性能 CPU 往往：**ISA 像 CISC，内部翻译成 micro-ops (微操作) 执行像 RISC**，兼有两者的优点。

单周期、多周期与流水线

Single-cycle：让指令都在 1 个 “长周期” 完成 —— 特点：**CPI = 1** 但周期很长，资源不复用。

> ⚠ lw 是 RV32I 中最慢的指令 (~5)，div 则是常见最慢的指令 (~10)，其余比较慢的还有 mul (~3)。最快的是 beq (后文会说它实际只走5个阶段中的 3 个)。

Multi-cycle：让指令分步走 —— 特点：CPI > 1 但周期变短，资源跨周期复用。

⚠ **Pipeline**：把**不同指令的不同阶段重叠起来做**。例如：第 1 条指令在执行的同时、第 2 条在译码、第 3 条在取指。**pipeline 可以视作 ILP 的基础形式**。特点：理想情况下接近 CPI = 1，资源复用。虽然 CPI 不如单周期，但更短周期导致更高吞吐率。

> RV32 最难 pipeline 的 integer divide。

Superscalar：一个周期能发射/完成多条指令，所以可能 **CPI < 1**。

逻辑设计基础

> 处理器说到底就是电路。

0 伏附近的低电压表示 0，1 伏附近的高电压表示 1。1 根电线传输 1-bit，多位数据用总线 (bus) 表示。

三大硬件成分

组合逻辑元件 (Combinational elements)：负责对输入做 (函数) 运算并输出，**无状态** (即记忆)。

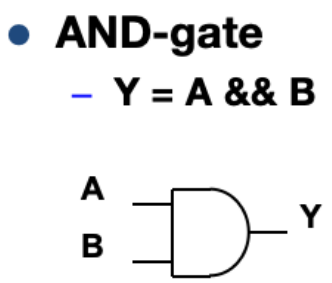
状态/时序逻辑元件 (State / Sequential elements)：负责存储状态信息 (如寄存器、存储器)。

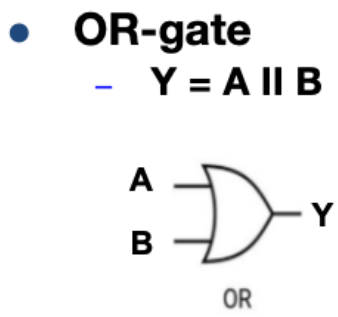
时钟 (Clock signals)：调节什么时候更新记忆元件。

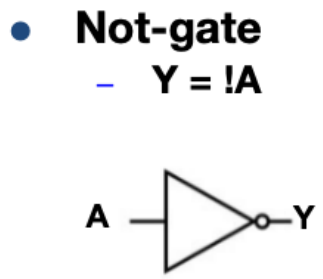
而处理器执行一条指令，本质上就是：

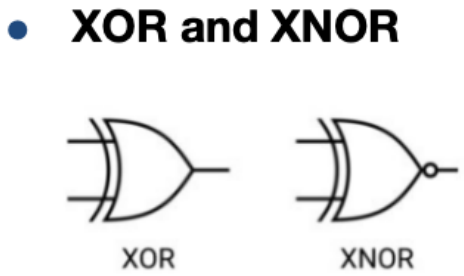
> 在一个周期内通过组合逻辑计算出 “下一状态”，然后在时钟即将进入下一周期时把它写进时序逻辑。

基础逻辑门



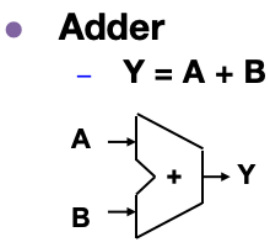


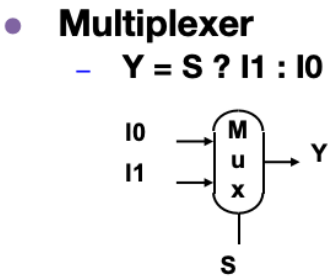


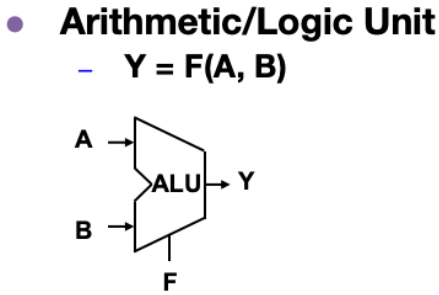


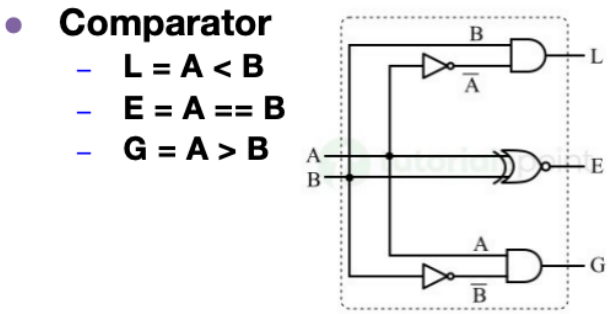
组合电路元件

- **MUX** (Multiplexer, 多路复用器)：`Y = S ? I1 : I0` (如果选择信号 S=1 输出 I1，否则输出 I0)，**三元运算逻辑**。“二选一” 或 “多选一” 问题，本质全靠 MUX (e.g.: ALU 第二输入到底选寄存器，还是立即数？PC 下一个值是 PC+4，还是 branch target?)
- **ALU**：`Y = F(A, B)` 由控制信号决定做 add/sub/and/or/compare…
- **Comparator**：**输出 L：A < B；E：A == B；G：A > B**。特别适合 **branch** 的条件判断。









组合电路始终在 “工作”，输入一变，输出就会在**延迟**后改变。(一个组合电路的延迟由**从任一输入到任一输出的最长逻辑路径**决定，这就是 **critical path**)

时序元件

- 寄存器 Regs：用来存储数据，依赖时钟信号上升边沿触发更新 (Clk 从 0 到 1 时)。
- Register with write control 多了写的权利：信号 Write = 1 正常更新；Write = 0 保持旧值不变。不是每条指令都需要写寄存器 —— ⚠ **对于 RV32I 的 Branch/Store 指令，Write 信号恒为 0**。

时钟

一个周期的要求是：

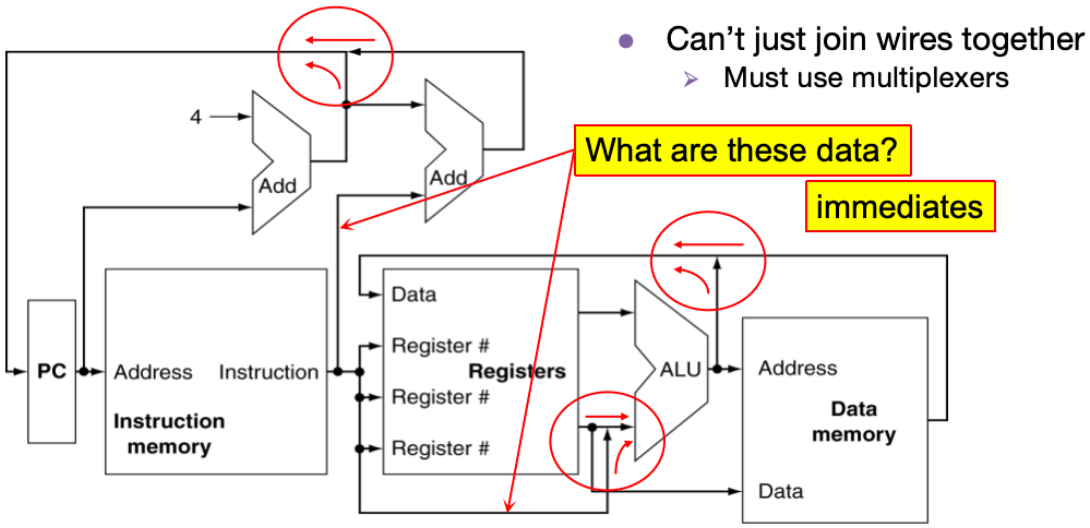
> 从某个状态元件输出开始，经过组合逻辑，必须在下一个时钟边沿之前稳定到达另一个状态元件输入。

时钟不能太快，否则数据未稳定就被写入产生错误，所以时钟周期的下限取决于**最长组合路径**。

RV32I 处理器的 “MVP 模型”

(图示为最简可运行的单周期 RV32I 处理器子集的模型)

⚠ 圆形岔口表示多线并行，而图中从左到右**三处**圈出来的岔口必须用 MUX —— **PCSrc** (要 Branch/Jump 吗？)、**ALU inSrc** (用立即数还是寄存器？)、**MemtoReg** (从内存还是 ALU 写回？)



- **R-type** 比如 `add x3, x1, x2`：同时并行 `PC += 4` (用左侧的 ADD) 和 PC 给 Instruction memory 传地址。后者拆开命令，**ALUSrc MUX** 选择寄存器输入，**ALUctr** 要求计算 `x1 + x2`，不用 Data Mem.，**MemtoReg MUX** 选择 **ALU** 输入，从 Data 接口写回寄存器 x3。

- **I-type** 比如 `addi x3, x1, 42`： **ALUSrc MUX** 选择 **Sign-extended Imm** 输入。其余与 R-type 类似。

- **Load** 比如 `lw x3, 8(x1)`：**ALUSrc MUX** 选择(**Sign-extend 12 to 32**) **8**，**ALUctr** 计算 `x1 + 8`，Data Mem. 读取计算结果对应的内存，**MemtoReg MUX** **选择 *Data Mem.** 输入，从 Data 接口写回寄存器 x3。

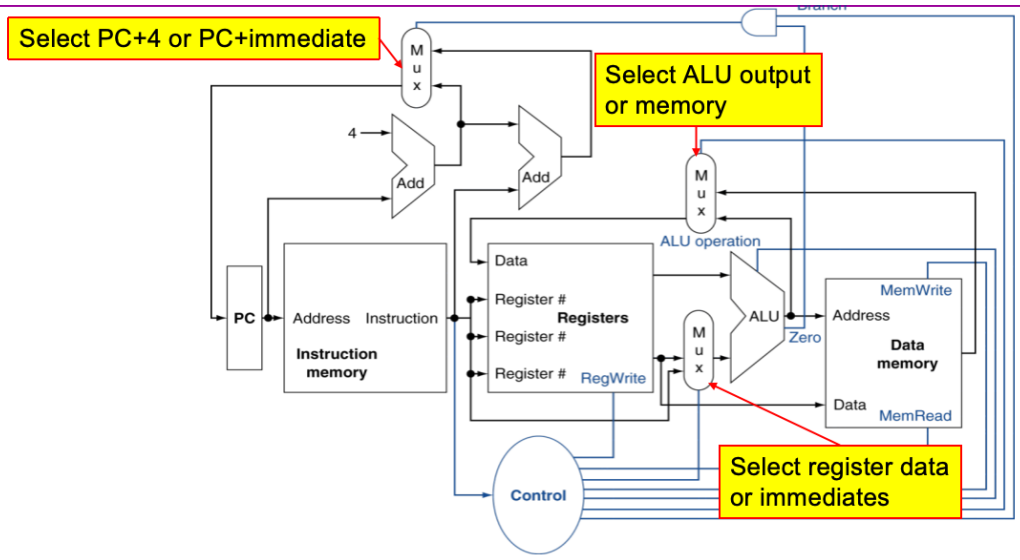
- **Store** 比如 `lw x3, 8(x1)`：反之把 x3 值写入 `data memory[x1 + 8]`，不写回数据。其余与 Load 类似。

- **Branch** 比如 `beq x1, x2, 88`：**ALUSrc MUX** 选择寄存器输入，**ALUctr 用 Comparator** 比较 x1 和 x2。同时计算 PC + 4 和 PC + 88，根据比较结果决定 **PCSrc** 取哪个。

- **Jump** 在 **MemtoReg** 处需要多一个 **WBsel** (Write Back Select) 元件，后续再讨论。

Control

- 1bit MUX 选择 PCSrc
- 1bit MUX 选择 MemtoReg (写回的是 ALU 值还是内存值)
- 1bit MUX 选择 ALUSrc
- 1bit Write Enable 控制是否做 Mem. read/write
- **4bit ALUctr** 控制 ALU 做 add/sub/and/or…



比如 `lw` ：ALUSrc = 1 (SE Imm); MemRd = 1 (需要读内存); MemtoReg = 1 (从内存写回寄存器); RegWr = 1 (需要写回寄存器)。

又比如 `sw` ：ALUSrc = 1;MemWr = 1; RegWr = 0。

How to Design a Processor ?

我们以单周期 RV32I 最小子集为例。

第 1 步：分析指令集需求

把每条指令的语义写成寄存器传送 (register transfer) 形式：

Inst	Register transfers
ADD	R[rd] <- R[rs1] + R[rs2]; PC <- PC + 4
SUB	R[rd] <- R[rs1] - R[rs2]; PC <- PC + 4
LOAD	R[rd] <- MEM[R[rs1] + sign_ext(Imm12)]; PC <- PC + 4
STORE	MEM[R[rs1] + sign_ext(Imm12)] <-R[rs2]; PC <- PC + 4
ADDI	R[rd] <- R[rs1] + sign_ext(Imm12)]; PC <- PC + 4
BEQ	if (R[rs1] == R[rs2]) then PC <- PC + sign_ext(Imm12)] else PC <- PC + 4

⚠ 总结出 datapath 至少需要：1、(Instruction) Memory；2、Register File (32 × 32 bit)；3、PC；4、Extender (立即数扩展)；5、ALU；6、用于 PC+4 或 PC+imm 的加法器。

指令	PC	Instr Mem	Reg File	Extender	ALU	Data Mem	PC+4 Adder	PC+imm Adder
R-type	1	1	R: rs1/rs2, W: rd	0	1	0	1	0
I-type	1	1	R: rs1, W: rd	1	1	1(lw) or 0(addi, jalr)	1	0
S-type	1	1	R: rs1/rs2	1	1	1	1	0
B-type	1	1	R: rs1/rs2	1	1 (ALU Comparator)	0	1	1
J-type	1	1	W: rd	1	0 (minimal realization)	0	1	1

第 2 步：选择数据通路组件与时钟方法

1、**CLA-Adder**：

- In a ripple-carry adder

$$C_i = (A_i * B_i) + (A_i * C_{i-1}) + (B_i * C_{i-1})$$

- In a carry lookahead adder

$$g_i = (A_i * B_i)$$

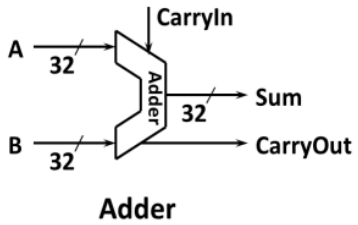
$$p_i = (A_i \text{ xor } B_i)$$

$$C_1 = g_0 + (p_0 * C_0)$$

$$C_2 = g_1 + (p_1 * g_0) + (p_1 * p_0 * C_0)$$

$$C_3 = g_2 + (p_2 * g_1) + (p_2 * p_1 * g_0) + (p_2 * p_1 * p_0 * C_0)$$

Now, the carry propagation chain disappears



2、**寄存器**：依赖 Clk (clock input) 信号作为 Write-Enable (1/0：有/无 “写权利”)

3、**寄存器文件**：32 个寄存器

- **Datapath**：**2 个 32-bit 读口** (Read) 总线 (busA, busB)、**1 个 32-bit 写口** (Write) 总线 (busW)；
- **Control**：3 个 **5-bit** 选择寄存器编号的线 RA、RB、RW (对应 ISA 的 rs1, rs2, rd)。
- 特殊的，2-wide superscalar 至少需要 4 个读口、FMA 需要 3 个读口。

4、**主存 (Memory)**

- **Datapath**：1 个 32-bit 读口总线、1 个 32-bit 写口总线；

- **Control**：32-bit Address、Write Enable (0/1)。

第 3 步：组装 datapath

1、取指模块 (**Instruction fetch unit**) 都是相似的，**“fetch”**：`mem[PC]` 和 **“update”**：`MUX: PC += 4 or PC += Imm` (next instruction 还是 branch?)

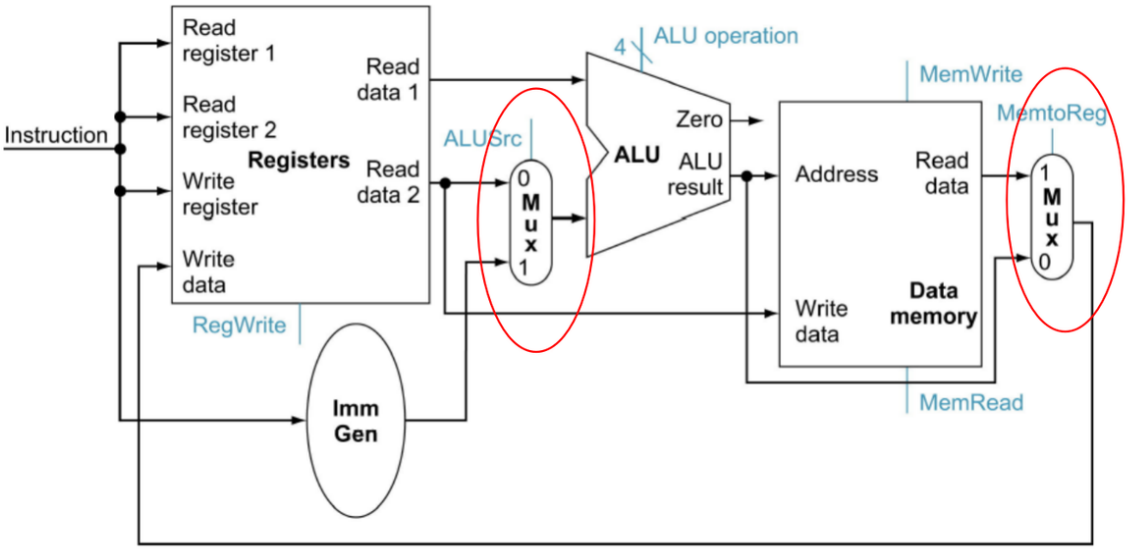
> 一个加法器够吗？

—— 理论上某些设计中可以复用，但在**单周期**里不够用，往往要同时计算 PC+4 与 PC+immediate。

> beq 为什么不能用 "`sub rs3, rs1, rs2 = 0`" 而要用 comparator？

—— overflow！如果一定要用减法比较，则要引入如 **V (overflow) / C (carry) flags** 辅助判断。

2、其余的执行模块应当用 MUX 集成在一个最小复用电路中：



> MUX 核心要义：**不要想着每条指令单独造一套电路，尽量共用数据通路，再用控制信号配置它。**

比如上图中典型地把内存类指令和 R-type 指令的 datapath 合并。

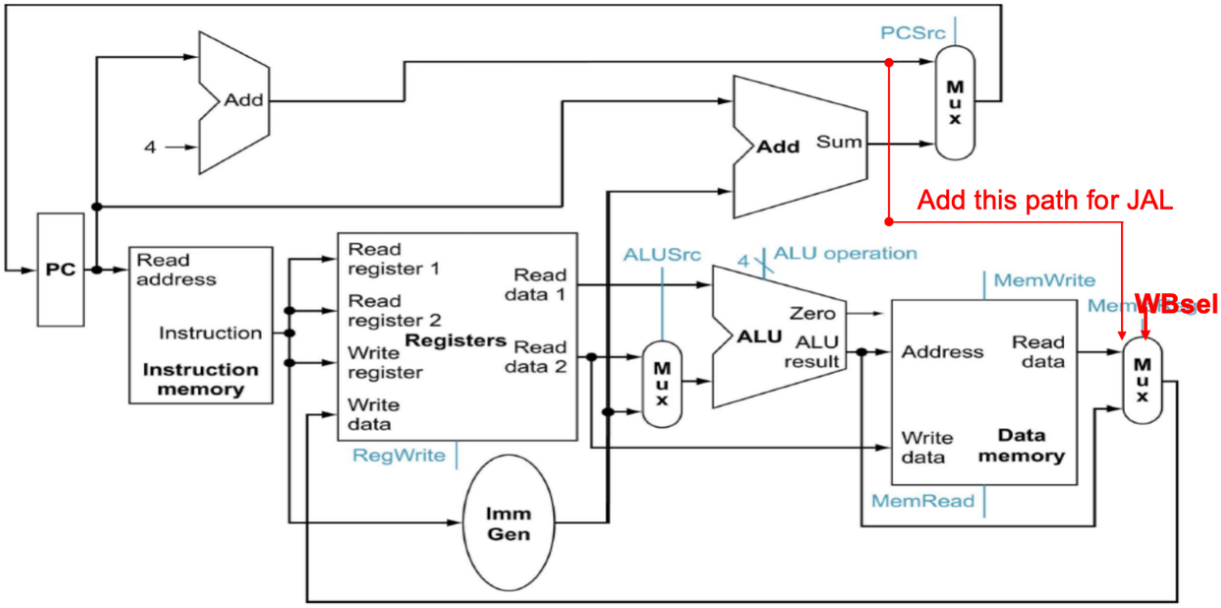
3、**上下文切换 (context switch) 本质上就是 “切换 state (状态)”**。

> **不是把机器里所有东西都存下来，而是把 “恢复这个执行流所必需的架构状态 (architectural state)” 保存/恢复** —— 包括 PC、Register Files(通用和特殊寄存器)、SP (栈指针)、RA (返回址)。

- **Thread switch 保存 processor registers**
- **Process switch 保存 PCB (包括 registers)**

> cache / control signals / pipeline state 等**微架构状态 (Microarchitectural state)** 不一定要保存

4、整体电路：(增加一条 `jal` 需要的通路：跳转到目标地址并把返回地址 `PC+4` 写入 rd)



5、时钟周期：<u>一个寄存器可以在同个周期里被读出，经过组合逻辑运算，再在同周期末写回。</u>

> 读发生在时钟边沿后电路传播的阶段；写发生在下一个时钟边沿前 (Clk 上升期)。

- “时钟周期的下限取决于**最长组合路径**。” —— 即 `lw` 的传播路径经过以下计算：

> Critical Path = "PC's Clk-to-Q" + "Instruction Memory Access Time" + "Register File Access Time" + “ALU 32-bit Addition Time” + “Data Memory Access Time" + "Register File Setup Time" + "Clock Skew".

(PC's Clk-to-Q：时钟到来后，PC 输出稳定需要一点时间。Setup time：写回寄存器前，输入必须提前稳定一段时间。Clock skew：时钟信号传递到不同部件有时差。)

- 所以单周期 CPU 的频率上限，通常被最慢路径 `lw` 卡死。

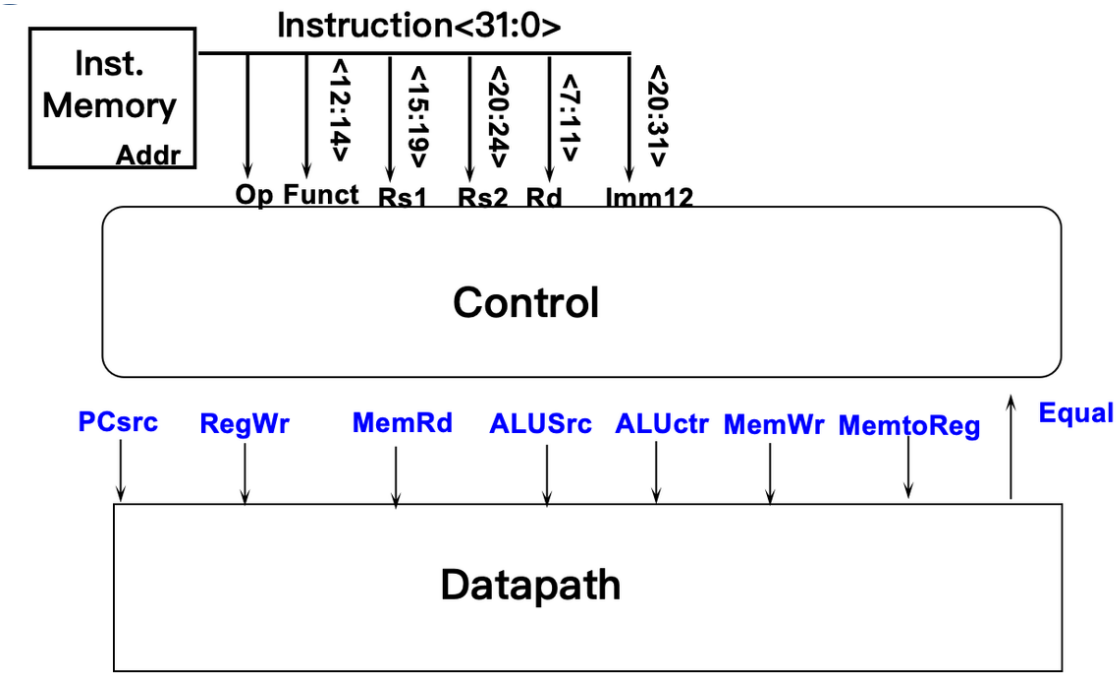
单周期处理器的明显缺陷

- 因为没有 ILP，资源不复用，需要多个冗余 Adder(PC 更新和 ALU 计算)；
- 同周期里不能反复经过同一组合逻辑/状态元件；
- 寄存器文件无需复制，但要多端口(2 read, 1 write)；
- `mul` 和 `div` 计算是迭代性的，得在单周期里 “重复展开”，开销非常昂贵。

所以更现实的方法是允许多周期执行，用 counter 控制重复使用同一硬件资源，代价是这些指令的 CPI 变大。

第 4 步：分析每条指令对应哪些控制点

确定每条指令在不同 MUX 处如何分类选择。



第 5 步：组装控制逻辑

实现从 opcode / funct 到控制信号的译码电路。

这五步就是从 ISA 到微架构的工程路线。

常用术语总表

信号	作用	常见含义
PCSel / PCSrc	决定下一条 PC	0=PC+4, 1=branch/jump target
ImmSel	决定立即数格式	I/S/B/U/J
RegWEn	是否写寄存器	1=写, 0=不写
BrUn	比较是否按无符号	1=unsigned, 0=signed
BrEq	比较结果：相等	rs1==rs2
BrLT	比较结果：小于	rs1<rs2
Asel	ALU A 输入来源	0=rs1, 1=PC
Bsel	ALU B 输入来源	0=rs2, 1=imm
ALUOp	ALU 粗分类	load/store, branch, R/I-type
ALUSel	ALU 具体运算	ADD/SUB/AND/OR/SLT
MemRW	数据内存读/写	Read / Write
WBSel	写回来源	0=mem, 1=alu, 2=pc+4

Pipelined Implementation

> 流水线 (pipelining) 的核心思想即**让多条指令的不同阶段“重叠执行”**。理想流水线是 “单发射” 的——即**每个 short cycle 开始 1 条指令**。

基础概念：

- 吞吐量 (Throughput)：平均多少周期完成一个指令？
- 指令延迟 (Latency)：单条指令从发射开始需要多少时间完成？
- 阶段 (Stage)：所有指令可以被分为多个互相硬件独立的阶段。
- Fill：指令依次被发射；Drain：等待最后一条指令的延迟。

⚠ **RISC-V ISA 天然适合流水线**：

- **指令长度固定 32-bit** (除了 "C-Extention" 先是 **16b**)，更易将指令划分成规律的阶段 (而 x86 可变长 1–17B)。
- **指令格式少且规则**：译码 + 读寄存器能在同一步完成。
- **L/S 架构**：只有 load/store 访存；地址计算在 EX，访存在 MEM，分工清晰。

五级流水、吞吐与耗时计算

经典 5 级流水：将指令划为五个阶段，分别占用处理器不同的区块

1. **IF**：取指（Instruction Fetch），包括 PC(内含 Clk)、ADD4、IMEM；

2. **ID**：译码 + 读寄存器（Instruction Decode & Register Read），包括 Regs、ImmGen(Sign-Extend)；
3. **EX**：执行/算地址（Execute / Address Calculate），包括 ALU，及其 Asel、Bsel；
4. **MEM**：访问数据存储器（Memory Access），包括 DMEM；
5. **WB**：写回寄存器（Write Back），包括四元计算器 WBsel。

> ⚠ 流水级别称为 “流水深度” (depth)，更深的流水的优点是更短的周期、更高理论频率和理论 CPI (都不一定成正比例提升)，缺点则是数据/控制风险的惩罚变大、需要更多流水寄存器、Control 变复杂。

单周期机最小周期为最慢指令 lw 的总时长 ("long cycle")；多周期机和流水线机最小周期为最慢阶段时长 ("short cycle")，比如 MEM/EX。周期 (ps) 的倒数为频率 (GHz)。

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
LW	IF	ID	EXE	MEM	WB			
ADD		IF	ID	EXE	MEM	WB		
BEQ			IF	ID	EXE	MEM	WB	
SUB				IF	ID	EXE	MEM	WB

- 如果 `1 long cycle = 5 short cycles`，LW 要 5sc (所有阶段都要)、SW/ADD/SUB/JAL 要 4sc (后三者没有 MEM，SW 没有 WB)、BEQ 要 3sc (没有 MEM 和 WB)。那么执行 `LW, LW, ADD, SUB, BEQ, SW, SW` 需要：
  - 单周期：“延迟向上对齐”—— `7lc = 35sc`，理想吞吐量 (**Throughput**) 为 1条指令 / 5sc；
  - 多周期：“实际需求”—— `5sc x 2 + 4sc x 4 + 3sc x 1 = 29sc`，理想吞吐量为 1 条指令 / 平均 CPI (sc)；
  - 流水线：“阶段向上对齐” **fill + drain** (等最后一条指令走完剩下4个阶段)—— `7sc + 4sc = 11sc`，理想吞吐量 (⚠ 不算 drain) 为 1 条指令 / sc (CPI≈1)。
- 故而，三者在五级流水下理想吞吐量为：**1/5、1/3~1/5、1**。

> ⚠ **单周期受到最慢指令延迟限制、多周期受到 CPI 限制、流水线受到最慢阶段限制。**

通用公式

(short cycle = n；流水线阶段数 = N)

- **单周期** (用最慢指令定周期)：$T = n \times T_{long}$。
- **多周期** (指令数为 i)：$T = \sum_i CPI(i)$。
- **理想 N 级流水**：$T = n + (N-1)$，即 **drain = latency - 1**。
- **流水的提速 (speedup) 公式**：$T_{pipe} = \frac{T_{SingleCycle}}{N}$，即提速 N 倍。(前提是各阶段平衡，如果各阶段耗时差异大，会导致提速下降)
- ⚠ 流水相对于多周期非流水的提速：$T_{pipe} = \frac{\sum_i CPI(i)}{i}$，即提速 “平均 CPI” 倍。

> ⚠ 流水线加速来自 **throughput 增加**，**而非 latency 减少**，事实上单条指令的 latency 反而可能更大 (若不平衡)。

流水线的代价

流水线要想“每个短周期都推进一条指令”，前提是——

- 每个阶段需要的硬件资源能同时服务不同指令；
- 否则就会出现**结构冒险 (structural hazard)**：资源冲突导致必须停顿（stall），流水线“断流”。

E.g.：如果 IF 和 MEM 共用同一个单端口内存，那么当某条 LW/SW 在 MEM 阶段访问内存时，下一条指令在 IF 阶段就必须等 (插入 bubble/stall)。解决需要**把内存资源翻倍** (**I-cache + D-cache** 分离，或多端口存储器)，增大硬件开销。

> ⚠ I, D Cache 分离**可以**防止结构风险、增大缓存访问带宽，但**不能**提升 hit-rate 或者提高缓存可靠性。

流水线的代价是硬件冗余或更复杂的控制，理想的 “CPI=1” 只在 “资源足够且结构冒险都被解决” 时成立。此问题在下文 (WEEK 9 部分) 展开讨论。

例题

- Assume individual stages of the datapath have the following latencies:

IF	ID	EX	MEM	WB
250ps	350ps	150ps	300ps	200ps

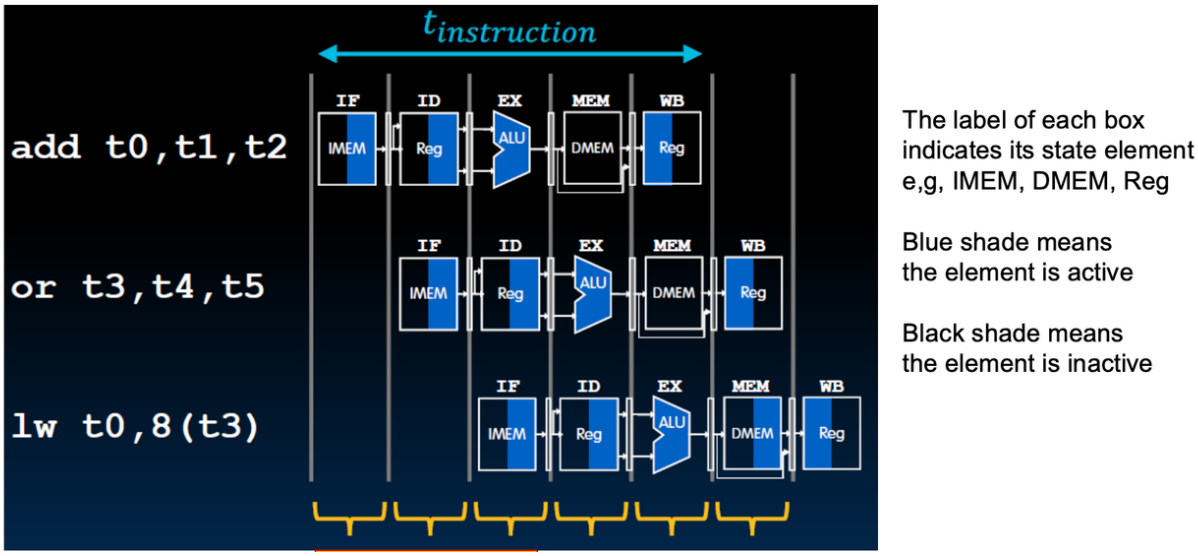
- Assume that instructions executed by the processor are broken down as follows:

alu	beq	lw	sw
45%	20%	20%	15%

- 时钟周期为？——1250ps (单周期非流水)，350ps (多周期非流水)，350ps (流水)。
- `lw` 命令的延迟为？——1250ps (非流水)，**350ps x 5 = 1750ps** (流水)。
- 如果改成六级流水，应该把哪个阶段拆开，拆开后周期为？——拆 ID，周期为 300ps。
- 提速？——1250/350 (相对于单周期)，60% x 4 + 20% x 5 + 20% x 3 = 4 (相对于多周期)。

流水线的实际实现

流水线的 “节拍” 由最慢阶段决定，不同于多周期非流水，流水线中所有指令不能跳过阶段，否则会打乱节拍。



- Sequentially 串行：同个指令的不同阶段，不能乱序；
- Simultaneously 并行：不同指令的不同阶段可以同时进行。

特殊的，不是所有指令的 EX 都可以在单个短周期内算完，比如 `div`，对于这种高延迟操作：

1、**Stall (停住流水线)**：下一个指令停在 EX 前面一个阶段，形成气泡 bubble。操作简单，但会导致吞吐和提速崩溃。

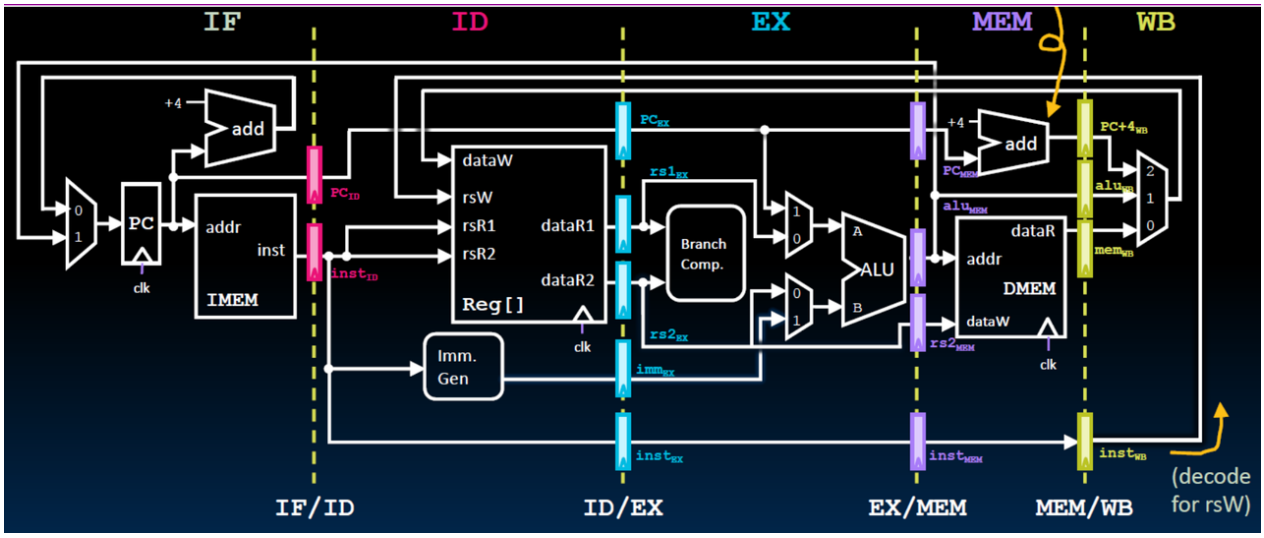
2、**Separate long-latency pipeline (另开长延迟功能单元)**：把 DIV / FP 这种操作交给专用单元 M-Extentions 和 F/D-Extentions，让主流水线能继续推进其它独立指令，这就是 RISC-V 分离模块的缘由。通常，指令在 WB 阶段**按程序顺序**退休，原因是 “异常/陷阱 (traps)” 语义：如果后面的指令先退休了，而前面的指令后来触发异常，会导致程序状态不一致。然而 RISC-V **避免了很多算术异常**，所以乱序退休也是可行的。

真正实现流水线的关键：流水寄存器

在单周期里，一条指令的控制信号“只服务这一条”，没歧义；但在流水线里，同一时刻有多条指令分别处于不同阶段，你就会遇到一些典型的混乱：

- “现在该用哪条指令的 rs1/rs2/imm？”
- “现在 WB 应该写哪条指令的 rd？”

解决方法就是：**在阶段之间插入 pipeline registers (IF/ID、ID/EX、EX/MEM、MEM/WB)**，把“数据 + 控制信号”一起往后带，保证每个阶段用到的都是“属于同一条指令”的那份信息。



> Pipeline registers **不是 architectural state**。上下文切换需要保存的是架构态（寄存器、PC、内存等），而流水寄存器、分支预测器、cache 等属于微架构态。

Pipeline Hazards

除去 Fill & Drain 的开销、阶段划分不平衡的限制，Pipelining 还面临着 **data** hazards，**structural** hazards 和 **control** hazards。三类 hazards 通常会使后续指令被卡在 **ID 阶段**，形成 **stall**，从而产生 **bubble**，效率下降。

Fill & Drain 的开销决定了 ⚠流水线偏爱长程序——

> 流水线的 fill/drain 都是额外成本，只有中间的完全并行时才是最优的。 流水线优化的是长程序，而对很短的代码段加速未必明显。

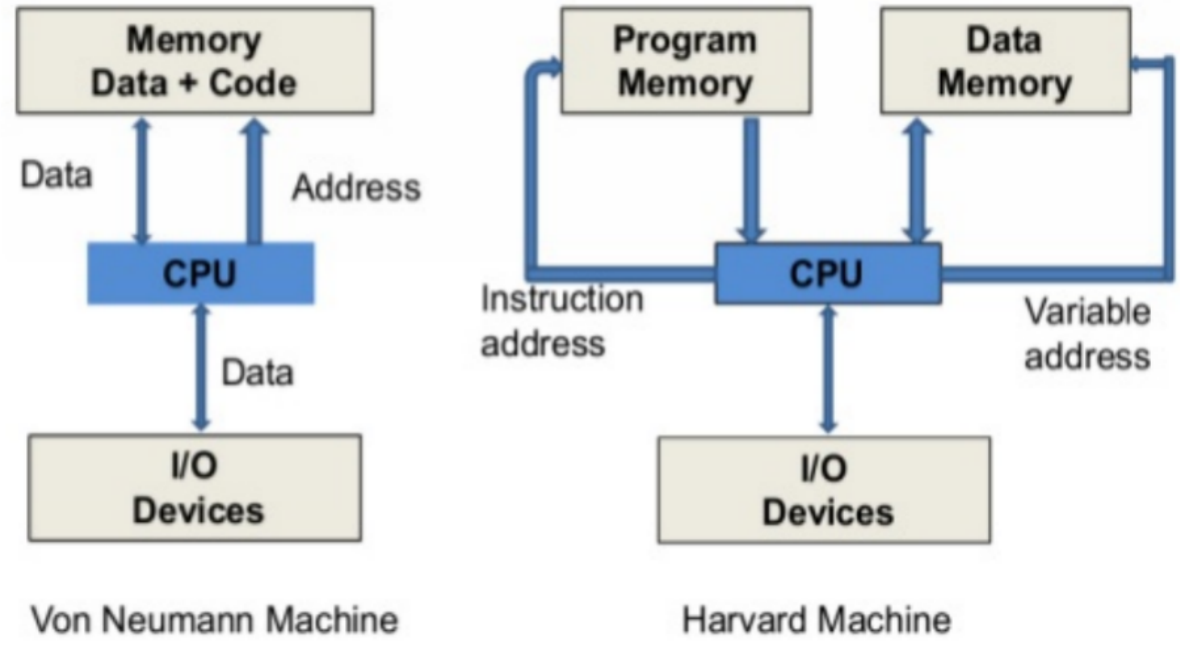
结构风险

> 结构风险是最朴素的资源冲突：硬件不够，导致两条指令同时争抢同一资源。

例子 A：IMEM 和 DMEM 端口冲突

- IF 阶段要取指令，要访问 **IMEM**；
- MEM 阶段的 load/store 要访问 **DMEM**。

如果机器采用的是 **Von Neumann architecture (冯·诺依曼结构)**，即指令和数据共用一套内存端口，那么当一条指令在 MEM 阶段访问数据时，另一条指令在 IF 阶段就没法同时取指了，于是冲突发生。



例子 B：两个相邻的除法指令

除法器往往不是 1 个周期做完，而是要占用功能单元很多周期，例如 32、64 个周期。如果两条除法指令连续出现，第二条除法就会发现：除法器还没腾出来。

规避结构风险

1、**Pipeline stalls**

最直接的方法就是冲突时让后继指令在 ID 阶段等着，插入 bubble。 简单但性能下降。

2、**增加资源**

在 **标量架构 (scalar architectures)** 中，结构冒险相对不常见，因为 —— 寄存器文件通常有足够的端口；有多个乘/除法单元或者这些功能单元本身就可流水化；IMEM 和 DMEM 通常分离；常有两个加法器：一个给 PC+4，一个给算术运算。拿硬件换性能。

3、**Instruction Reordering**

可以由：编译器静态调度 (static scheduling) 或硬件动态调度 (dynamic scheduling) 来尽量错开那些会争资源的指令。 拿顺序安排换性能。

4、**ISA design**

某些指令形式会额外要求更多写端口，比如 post-increment。或直接在 ISA 层禁止某些容易引发冲突的设计，例如 ALU 指令禁用 MEM。

- 比如 R-Type 不存在 MEM 阶段，但不能提早 WB 打乱节拍，导致结构风险。

数据风险

> 数据冒险是指**后继的指令依赖前序指令产生的数据，但数据还没准备好。**即 **RAW** 依赖。

比如遍历顺序数据结构 (链表)；

再比如：

`add x9, x0, x1` ：结果在 EXE 末尾算出来，在 WB 阶段写回 `x9`

`sub x10, x9, x3` ：它需要在 ID 阶段读 `x9`

当 sub 到 ID 时，add 可能还没把 x9 正式写回寄存器堆。最糟糕时，就只能让 `sub` 等到 `add` 写回完成，形成 2 个周期的 bubble。

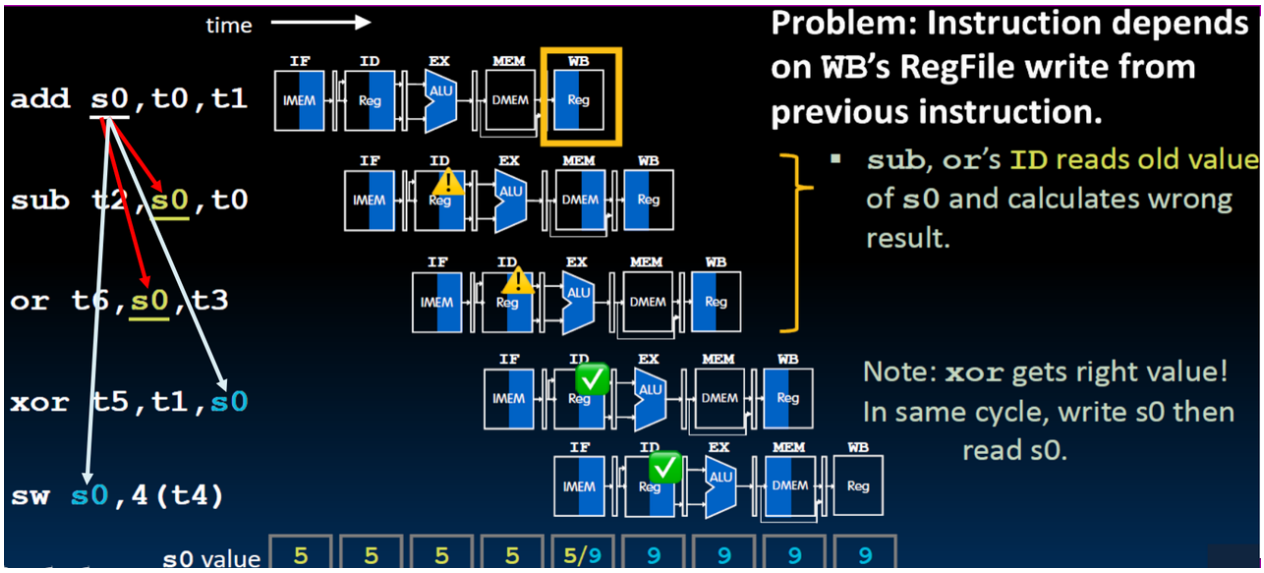
数据风险可以大致分为三类：

Register Access Issues

如果写和读同个寄存器恰好发生在同一周期，我们显然只需**同周期先写后读 (write-then-read) **解决。不必 stall。

ALU Result Access Issues

下图中， `xor` 指令的 Register Access Issue 被 Write-then-read 解决了，但 `sub` 和 `or` 因为 `add` 尚未 WB 得到错误值。



硬件-方案：Pipeline Stall at ID stage

> 按照上文中的例子那样，紧接着的下条指令 **stall 2 cycles**。

这种方法属于 **Interlocking** …… Which is terrible.

硬件-优化方案：Data Forwarding (Data Bypassing)

> 为了减少数据风险的惩罚，可以不等写回寄存器堆再读，直接把依赖对象流水中已经算出来的结果 “旁路” 送给 ALU。

能成功是因为后续指令进行到 EX 时，前序依赖**至少**已经结束了 EX 进入 MEM，所以可以直接从那里拿一份值。

这是现代流水线解决 ALU 数据冒险的核心机制之一。

Load Issues

> Load 产生的数据在 MEM 阶段缓慢的访存后得到，⚠ 紧接着的命令若依赖**必须 Stall 1 cycle**。

现实中，Load DMEM 也有 Latency，需要的时间与数据从哪里来有关：Cache 2 需要 2~3 cycle；Cache 3 需要 20 cycle；DRAM 需要 200 cycle。

数据风险的检测

> 前面流水级里将要写回的目标寄存器 RegRd，是否正好等于当前指令将要读取的源寄存器 RegRs？

从 EX/MEM 流水寄存器 Bypass：

- EX/MEM.RegRd = ID/EX.RegRs1
- EX/MEM.RegRd = ID/EX.RegRs2

从 MEM/WB 流水寄存器 Bypass：

- MEM/WB.RegRd = ID/EX.RegRs1
- MEM/WB.RegRd = ID/EX.RegRs2

额外判断：

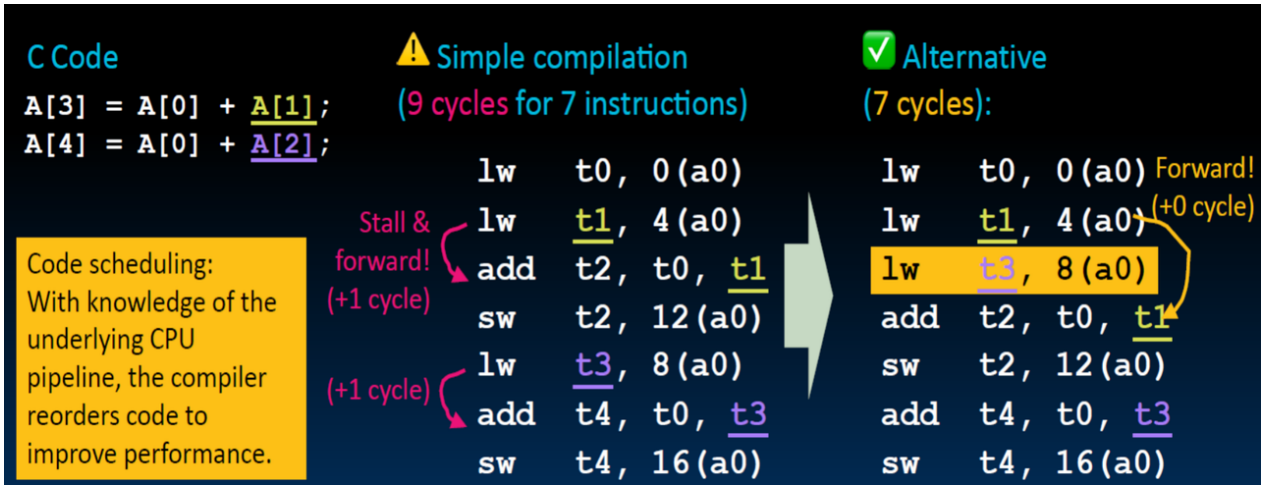
- 如果前面的指令**根本不写寄存器**，就不 forward。**所以要检查 RegWr**；
- 如果目标寄存器是 $0，也不 forward，因为零号寄存器恒为 0。**所以要检查 RegRd ≠ 0**。

MIPS (软件-优化方案)

> **MIPS = Micro-Processor without Interlocked Pipe Stages**——硬件尽量不做复杂的流水线 interlock，数据冒险交给编译器。

这就是 code scheduling：编译器乱序 (**OoO**) 重排指令，避开 **WAW**、**WAR**、**RAW** 依赖，如果找不到合适指令填空，就插入 NOP。非 MIPS 机中 code scheduling 亦可用于减少 Interlock，从而减少 Stall。

- **理论上有 “1 (最优) ~ 指令数的阶乘 (最劣)” 种排列**；
- ⚠ 重排的缺点：内存依赖不明确 (aliasing)、内存访问延迟未知且不确定
- ⚠ MIPS 的 Compatibility and Scalability 不足导致其普及不太成功。



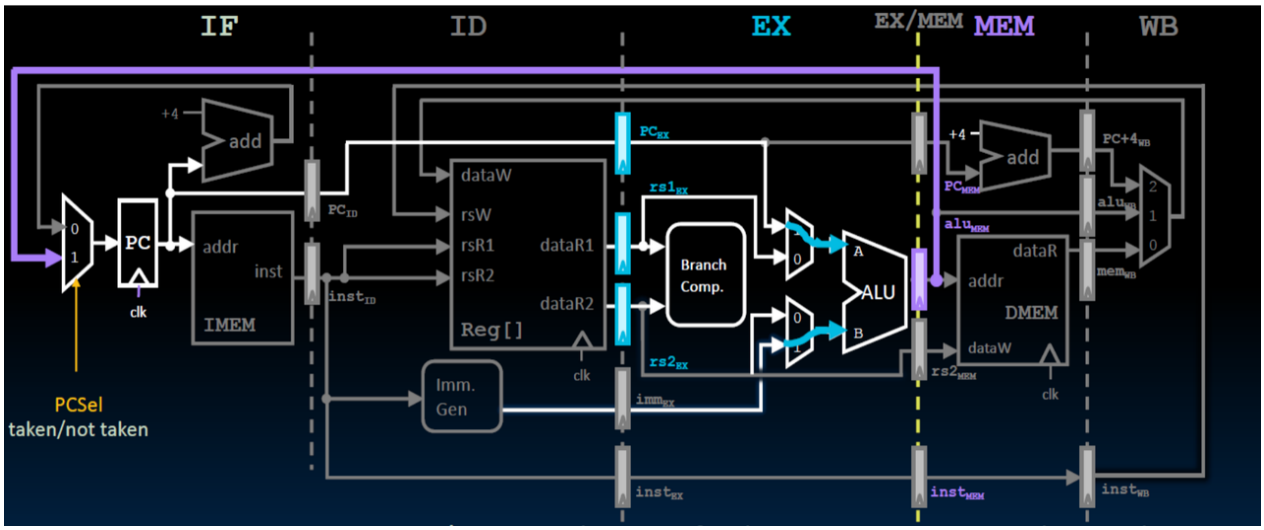
控制风险

> 在分支条件尚未判定 **T/NT** 或者跳跃地址尚未算出前，下次发射 IF 时并不知道真正应该执行谁。

- 控制风险来自：条件分支指令 (branch) 或 跳转指令 (`jal` or `jalr`)。
- 分支指令比例约占 **15% 到 30%**。约每 **4 到 5 条** 指令就有一条分支。意味着控制风险是**流水线性能的关键之一**。
- ⚠ 最难预测的控制风险是 "**indirect jump for branch JALR**"。

⚠ 控制风险的原因不是 “跳转” 这个行为本身，而在于直到 **EX** 这一步计算 (ALU/Comparator) 之后 (**MEM**) 才知道下个流水级真正的 IF 对象。故 `jal` 和 `jalr` **都会**产生控制风险。

我们先讨论 Branch 的 T/NT 问题。



- 如果不做风险防护，在 Branch 的风险发生期间，最多可能紧接着执行 **3** 个错误命令 (即 branch/jalr 执行 ID、EX、MEM **三个阶段**发射的三条指令)。
- 如果我们直接 **flush** 所有错误命令：20% 的指令是 branch，其中 50% 执行跳跃，基础 CPI = 1，flush 3 条指令 —— 导致至少 3 × 0.2 × 0.5 = **30% CPI Penalty**。(实际在 30~60%)

- 如果在 EX 阶段就知道分支结果 (比如 bypass 出来)，flush 的代价变为 2 条指令： 2 × 0.2 × 0.5 = **20% CPI Penalty** (实际在 20~50%)。所以——

> **分支预测结果越早决定越好，⚠ 最好在 IF 阶段。**

方案一：Predict Not Taken (原始)

最简单的预测方法——假设所有分支都不跳转，即默认下一条 PC = PC + 4，再在 MEM 阶段验证。“理论上” 性能损失减少一半。⚠ 但是通常代码中 **Branch 由 Loop (循环) 为主**——每个循环 T 或有好多次，但 NT 只有一次，实际上平均准确率约 **30~40%**，**CPI Penalty 约 28%**。

方案二：Static Branch Prediction (落后)

> 静态预测：预测结果在编译期就基本决定好，运行时不怎么变。

比如：

- **回跳 (backward branch) 预测为 T**，因为循环 (Loop) 回跳大多在迭代，都会成立，到循环结束时也只会失败一次；

  比如 `beq r3, r4, -8` → predict taken。

- **前跳 (forward branch) 预测为 NT**，因为编译器常把 "**then**" 部分放在顺序路径 (if, then, else…)；

  比如 `beq r3, r4, 8` → predict fall-through。

- 记录预测结果：ISA 可以为 branch 指令预留 prediction bit；或者像 HP 用指令中寄存器顺序表示 (beq r3, r4, 8 表示 NT / beq r4, r3, 8 表示 T，⚠ 其它不对称 Branch 不能直接交换寄存器顺序)

- 间接跳转 `jalr` 则很难用静态模型预测

但无论如何，这种静态预测准确率通常只有 **60%~70%**。编译器分析产生的静态预测约 **80~85%**

方案三：Dynamic Branch Prediction (当代)

> 在**运行时**根据同个指令的历史行为不断学习和更新的预测方法，准确率常能到 **90%–99%**。

BHT：Branch History Table

用最近 branch 指令的地址 (tag) 来索引，表里存分支过去的结果。⚠ **IF 阶段**查表，按记录预测下一次结果，**没有记录默认 NT**；再根据实际结果更新 BHT。

⚠ 表项里历史信息可能只要 1~2 bit，但 tag 可能要 30–46 bit。所以实际中往往直接拿 **PC 的低若干位** 去索引 BHT，而不做完整 tag。这会带来一个现象：**aliasing (别名冲突)**。不同分支可能映射到同一个 BHT 表项，互相污染历史。虽然 untagged BHT **简单、快速、低能耗、高性价比**，但高性能处理器倾向于用 tagged BHT 来减少 aliasing。

1-bit Predictor

(上次 T，这次就预测 T，vice versa)

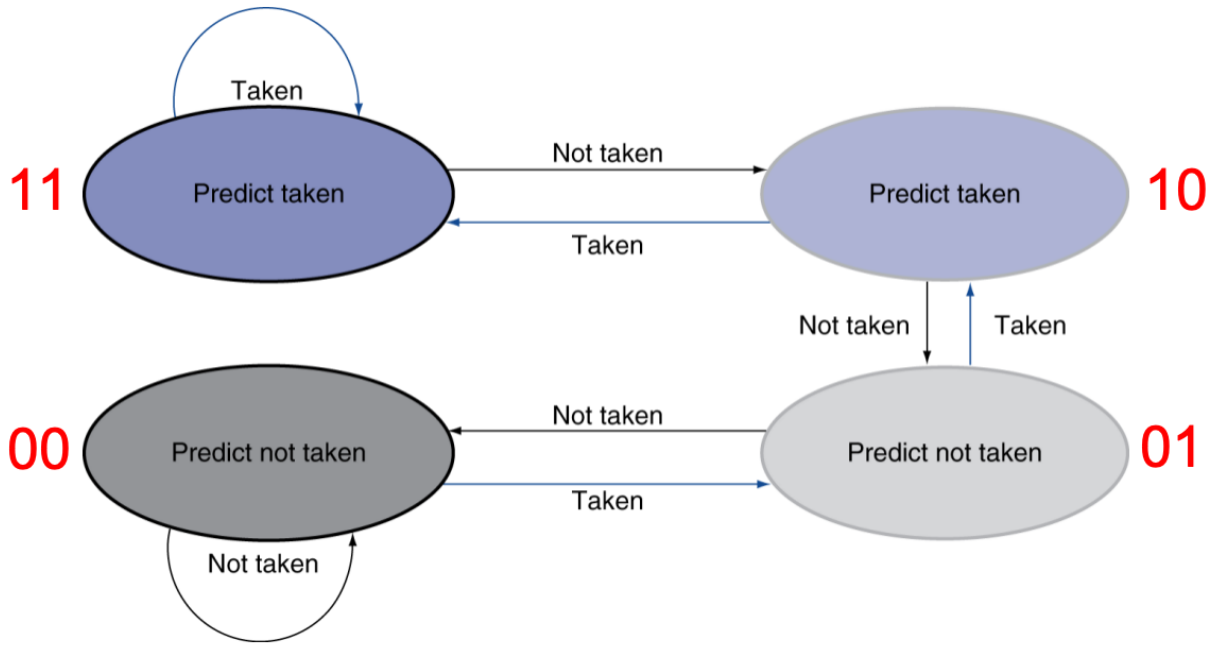
⚠ 它在处理嵌套循环 (**Nested Loop**) 时发生典型的 “双连错”，准确率约 **70~80%**：

1. 内层循环大部分都在迭代，都是 T；
2. 最后一次退出循环时，真实结果突然变成 NT，于是第一次误判，更新成 NT；

3. 下次再进入内层循环时，第一轮其实应该 T，但预测器还记着 NT，于是第二次误判。

2-bit Predictor (FSM)

> 只有连续两次预测错，才改变预测倾向。



这是一个**有限状态机 (finite-state machine)**。准确率约 **85~95%**

方案四：Correlated Prediction (最新)

也叫 **2-level adaptive branch prediction**，它不是只看 “这个分支自己上次怎样”，而是看：

> **最近若干个分支的整体模式，是否能帮助预测当前分支。准确率约 94~96%**

1. GHB：Global Branch History Register

GHB 是一个移位寄存器，记录最近几个 branch 的结果。每个 bit 表示一次 branch T/NT。每来一个新分支结果，就 shift 一次，把旧的出队。相当于记录最近控制流的上下文。

2. PHT：Pattern History Table

⚠ 用 Current PC 和 GHB 作为预测表的索引

PHT 里放的是 **2-bit saturating counters**。GHB 的内容作为其索引，如果当前 branch 实际 T，就把对应计数器 +1，否则 -1。它反映的不是 “某条 branch 永远怎样”，而是在某种全局历史下，这条 branch 倾向于怎样。

比如：

```
if (a==2) a=0;   // B1
if (b==2) b=0;   // B2
if (a!=b) {...}  // B3
```

如果 B1 T，B2 也 T，说明 a 和 b 都被置 0，那么 B3 很可能 **not taken**。B3 的结果和它前面两个分支的结果存在相关性 (correlation)。

> 准确率从 98% 到 99% 的提升不是 1% 而是 100%，因为性能由预测失败率主导，而非预测成功率主导 (长程序中失败代价高)。

BTB: 光顾着跳不跳了，你到底要跳哪里去？

即使有各种 Predictor，跳转时仍可能有 **1～2-cycle penalty**，取决于 target address 是在 ID 还是 EXE 阶段算出来——

> 分支预测有两个问题：Whether (跳不跳) & **Where (跳到哪)**。

为了解决“where”的问题，引入 **BTB** (Branch Target Buffer)，它本质上是 **target address 的 cache。**

- target address 通常不变，因为 branch immediate 不变；
- 因此可按 PC 建一个小缓存。在 IF 阶段取指时，用 PC 查 BTB；
- 如果命中，并且预测为 taken，就可以立刻去取 target 指令，而不用等后面阶段再慢慢算地址 。

⚠ 这就是为何现代处理器常把 BHT 和 BTB 配合使用：**BHT 负责 whether，BTB 负责 where**。两者共同减少 Control Hazard 的惩罚。

流水线中控制信号如何流动？

> ⚠ **Pipeline Regs, BHT, BTB 都属于 Microarchitecture**

EX stage

数据通路要有：rs1 内容、rs2 内容、PC、Imm 值

控制信号有：Asel、Bsel、ALUsel (所以需要两个 Adder)

MEM stage

数据通路里需要：ALUout (作为地址)、rs2 内容 (store 时要写入内存的数据)

控制信号有：MemRW、BrLt、BrEq

WB stage

数据通路里可能写回的来源有：ALUout、PC+4 (给 link register)、MEMout

控制信号有：**Wbsel、rd、WRen**

延迟使用：

- EX 用的控制信号，**1 个周期后** 才真正用；
- MEM 用的控制信号，是 **2 个周期后**；
- WB 用的控制信号，是 **3 个周期后**；
- RD (rd) 也要一路传到 WB 阶段。

当一条指令在 ID 阶段被译码出来时，控制单元已经知道将来要做什么，但它此刻还没到 EX/MEM/WB。所以这些控制位不能立刻消失，必须随同该指令一起存进**流水寄存器**，一级级往后传。